OFFRE SPÉCIALE : inscrivez jusqu'à 4 enfants pour le prix d'1 avec le code promo HAPPYFAMILLE23 !

Comprendre et utiliser l'algorithme des k plus proches voisins

Objectif

Comprendre l'algorithme des *k* plus proches voisins en intelligence artificielle (IA).

Points clés

- L'algorithme des *k* plus proches voisins permet de prédire à quelle famille une nouvelle entrée (une nouvelle donnée) va appartenir. Il calcule pour cela les distances de la nouvelle entrée au jeu de donnée, c'est-à-dire la distance entre la nouvelle donnée et toutes celles déjà présentes dans le jeu de données.
- Il s'agit d'un modèle de prédiction utilisé en intelligence artificielle pour la reconnaissance de formes mais aussi pour certains diagnostics médicaux.

Pour bien comprendre

- Écrire une fonction.
- Utiliser des bibliothèques.
- Utiliser la bibliothèque Matplotlib de Python pour créer un graphique.

1. Le principe de l'algorithme

a. Présentation de l'algorithme

L'algorithme des *k* plus proches voisins est un algorithme d'apprentissage automatique qui est qualifié de supervisé. Il s'agit de montrer à une machine un grand nombre d'exemples similaires afin de lui apprendre à résoudre certains problèmes.

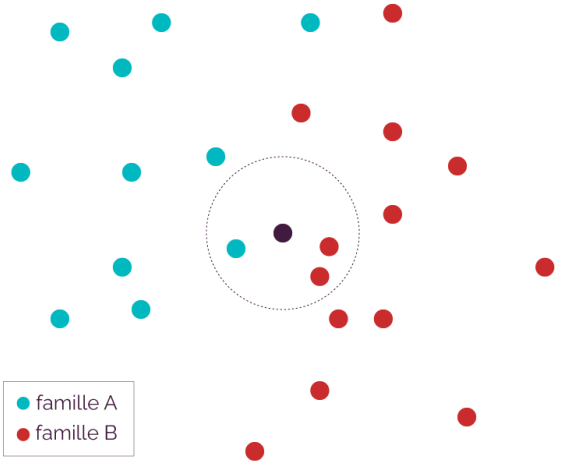
L'algorithme des *k* plus proches voisins permet de classifier des données de manière **artificielle** : c'est le programme qui détermine à quelle groupe (famille) appartient une nouvelle donnée entrée, en s'appuyant sur des données déjà entrées qui ont déjà été classées par groupes (familles).

b. Le fonctionnement de l'algorithme

1. On définit en entrée de cet algorithme un ensemble de données déjà classifiées (appelé jeu de données), **une distance *d*** et **un nombre entier *k***.
2. L'algorithme des *k* plus proches voisins calcule la distance entre toutes les données déjà classifiées et la nouvelle donnée qui vient d'être entrée.
3. L'algorithme extrait ensuite les *k* données déjà classifiées les plus « proches » de la nouvelle donnée entrée, c'est-à-dire les données déjà classifiées qui ont la distance *d* la plus petite avec la nouvelle donnée entrée.
4. L'algorithme choisit enfin à quelle famille appartient la nouvelle donnée, en cherchant la famille majoritaire parmi les *k* données identifiées.

Remarque
Cet algorithme se nomme *k*-NN, diminutif de ***k* Nearest Neighbors** : on le nomme l'algorithme des *k* plus proches voisins en français.

Exemple
On a un jeu de données qui permet de classer des individus dans deux familles A et B. On ajoute un individu en noir. On prend $k = 3$.



En appliquant l'algorithme *k*-NN, l'individu fera parti de la famille B : parmi ses 3 plus proches voisins, deux sont en effet rouges.

2. Les distances utilisées

On peut utiliser différentes distances entre les données, les plus usitées sont la **distance euclidienne** et la **distance Manhattan**.

Une donnée D_1 est constituée de n éléments que l'on considère comme ses coordonnées, on note cela par $D_1(x_1, x_2, \dots, x_n)$. On a de même $D_2(y_1, y_2, \dots, y_n)$.

Distance euclidienne

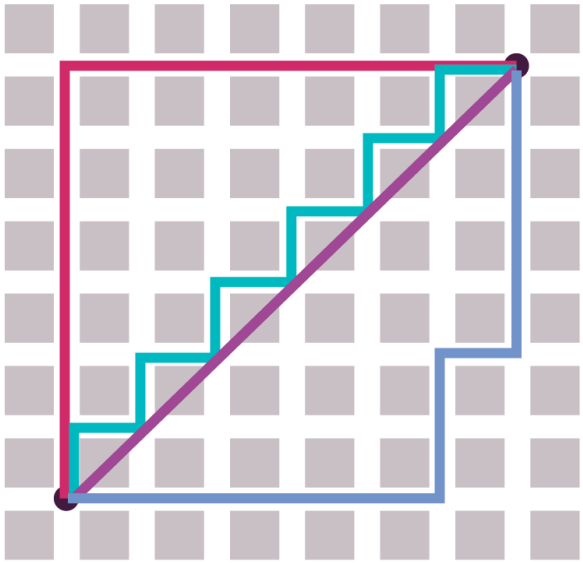
> La distance euclidienne est la distance utilisée pour calculer la distance entre deux points.

La distance euclidienne d entre les points D_1 et D_2 est donnée par la relation suivante.

$$d(D_1, D_2) = \sqrt{\sum_{k=1}^{n} (y_k - x_k)^2}$$

Distance de Manhattan d
La distance de Manhattan est nommée ainsi car elle permet de mesurer la distance parcourue entre deux points par une voiture dans une ville où les rues sont agencées selon un quadrillage.

> Exemple
> Sur le visuel ci-dessous, le tracé violet correspond à la distance euclidienne, tandis que les tracés rose, bleu clair et bleu foncé correspondent à la distance de Manhattan.



La distance de Manhattan d entre deux données D_1 et D_2 est donnée par la relation suivante.

$$d(D_1, D_2) = \sum_{k=1}^{n} |y_k - x_k|$$

On va prioritairement utiliser la distance euclidienne.

3. Ouvrir et lire un jeu de données

La difficulté consiste à utiliser les données déjà classifiées car le jeu de données est généralement dans un format CSV. Pour programmer les fonctions distances, il faut ouvrir le fichier et créer une liste.

import csv	On importe la bibliothèque CSV,
from math import*	pour utiliser la racine carrée qui appartient au module math.
with open('donnees.csv', 'rt',newline=" ") as fichier:	On ouvre le fichier donnees.csv. rt signifie avec le droit de lecture et en mode texte. La nouvelle ligne est symbolisée par l'espace. On lui donne le nom de « fichier ».
lecteurCSV=csv.reader (fichier,delimiter=",")	On utilise le lecteur de données csv sur le fichier avec comme délimiteur la virgule.
tableau=[]	On crée un tableau vide.
for ligne in lecteurCSV:	Pour chaque ligne,

tableau.append(ligne)	on place la ligne dans le tableau.
fichier.close()	Il faut toujours fermer le fichier !

Soit un jeu de données qui a *m* données. Pour calculer la distance euclidienne *d* entre le *i*-ème élément du jeu de données et la nouvelle entrée, on doit taper les lignes de code Python suivantes sachant que la nouvelle entrée est un tableau de longueur *m*.

d=0	On initialise la distance *d* à 0.
for j in range(1,m):	Pour *j* de 1 à *m*,
d=d+eval(tableau[i][j] - nouvelle[j])**2	on ajoute à *d* les distances respectives au carré.
d=sqrt(d)	Pour obtenir la distance euclidienne, on prend la racine carrée de *d*.

La programmation de l'algorithme est très technique, on utilise donc une bibliothèque spécifique qui contient tous les outils nécessaires à l'intelligence artificielle.

4. Utiliser l'algorithme - Exemple des iris

a. Présentation de la bibliothèque Scikit-Learn

> Scikit-Learn est une bibliothèque libre Python qui contient des jeux de données, ainsi que tous les outils et bibliothèques nécessaires pour l'intelligence artificielle. On la nomme en abrégé **sklearn**.

Cette bibliothèque contient un ensemble de jeux de données contenus dans **datasets**.
Elle contient également un package **sklearn.neighbors** qui contient tous les outils pour faire de l'apprentissage supervisé avec l'algorithme *k*-NN, en particulier l'outil **KNeighborsClassifier** qui permet de prédire l'appartenance d'une nouvelle donnée à une famille.

Voici les lignes de code à utiliser pour importer ces outils.

```
1 from sklearn import datasets
2 from sklearn.neighbors import KNeighborsClassifier
```

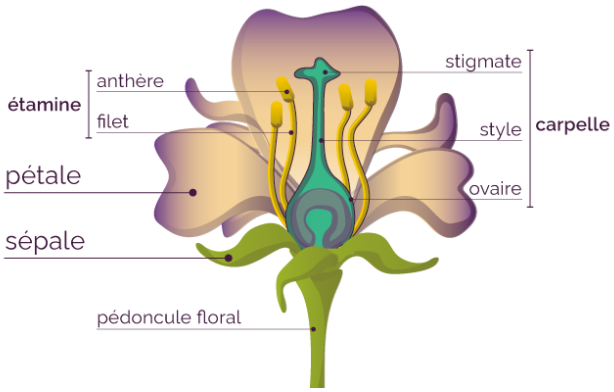
Voici l'explication ligne par ligne.

from sklearn import datasets	On importe le jeu de données datasets du module

	sklearn.
from sklearn.neighbors import KNeighborsClassifier	On importe le module de classification KNeighborsClassifier du module sklearn.neighbors.

b. Chargement d'un jeu de données

En 1936, M. Fisher a étudié les iris de Gaspesie, au Québec. Ces plantes comportent trois familles : Setosa, Versicolore et Verginica. Il a étudié la longueur des sépales et pétales pour 150 iris, ce qui a donné naissance au jeu de données Iris, aussi appelé Iris de Fisher.



Coupe schématique d'une fleur

Chaque fleur comporte ainsi des attributs (longueurs et largeurs des sépales, longueurs et largeurs des pétales) ainsi qu'une classe (0 pour Setosa, 1 pour Versicolore et 2 pour Verginica), qui sont répertoriés dans le jeu de données Iris.

La bibliothèque dataset contient ce jeu de données. Pour le charger dans un programme, il faut taper la ligne de code suivante.

```
irisData=datasets.load_iris()
```

c. Visualisation d'un jeu de données datasets

Pour visualiser les données, on utilise la bibliothèque Matplotlib, laquelle permet de tracer et de visualiser des données sous forme de graphiques.

Il faut pour cela taper les lignes de code suivantes.

```
import matplotlib.pyplot as pl
import matplotlib
```

Voici l'explication ligne par ligne.

import matplotlib.pyplot as pl	On importe matplotlib.pyplot avec un alias pl afin d'obtenir un environnement de travail.

import matplotlib	On importe matplotlib, pour pouvoir réaliser les tracés.

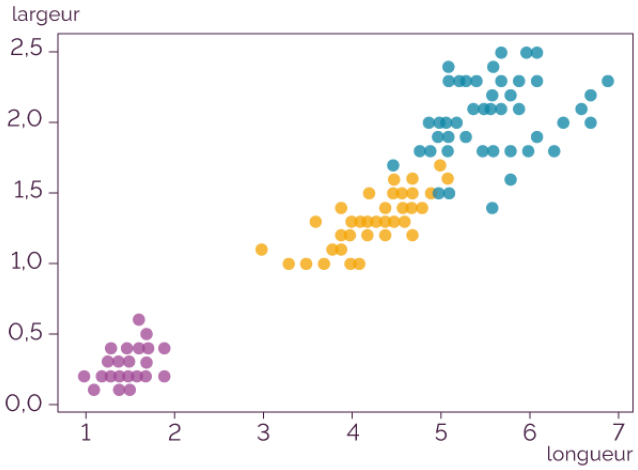
On va représenter la longueur et la largeur des pétales. Les points violets représentent les iris Setosa, les jaunes représentent les Versicolore et les bleus les Verginica. Voici les lignes de code Python.

```
6 clist=['violet','yellow','blue']
7 colors=[clist[c] for c in irisData.target]
8 pl.scatter(irisData.data[:,2],irisData.data[:,3],c=colors)
9 pl.xlabel('longueur')
10 pl.ylabel('largeur')
```

Voici l'explication ligne par ligne.

clist=['violet' , 'yellow' , 'blue']	Création de la liste des couleurs du graphique.
colores=[clist[c] for c in irisData.target]	Création de la liste des couleurs des 150 iris du jeu de données.
pl.scatter(irisData.data[:,2], irisData.data[:,3],c=colors)	Création du nuage de points de coordonnées (irisData.data[:,2], irisData.data[:,3]) avec la couleur associé.
pl.xlabel('longueur')	Ajout de la légende « longueur » sur l'axe des abscisses.
pl.ylabel('largeur')	Ajout de la légende « largeur » sur l'axe des ordonnées.

Ces lignes de code permettent de visualiser les données sur le graphique ci-dessous.



d. Ajout d'une entrée et prédiction

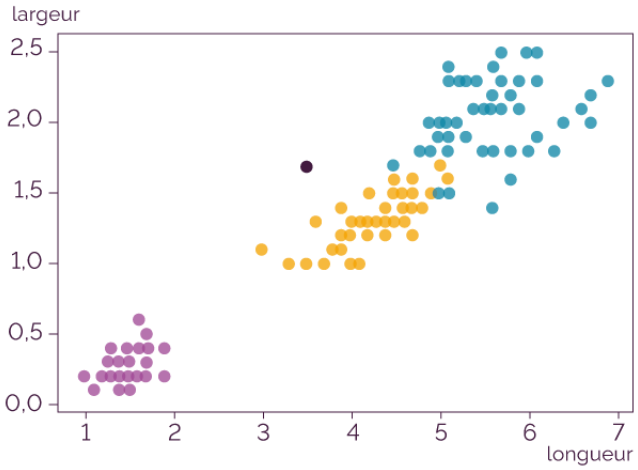
On s'intéresse à une iris ayant une longueur de pétale de 3,5 cm et une largeur de pétale de 1,7 cm. On souhaite déterminer à quelle famille d'iris cette plante appartient.

On ajoute pour cela la ligne de code ci-dessous à la fin du programme déjà existant.

```
pl.scatter(3.5,1.7,color='k')
```

Cette ligne indique qu'on ajoute au nuage de points le point de coordonnées (3.5,1.7) avec la couleur dont le code est 'k', c'est du noir.

On obtient le graphique suivant, où le point noir correspond à l'iris étudié.



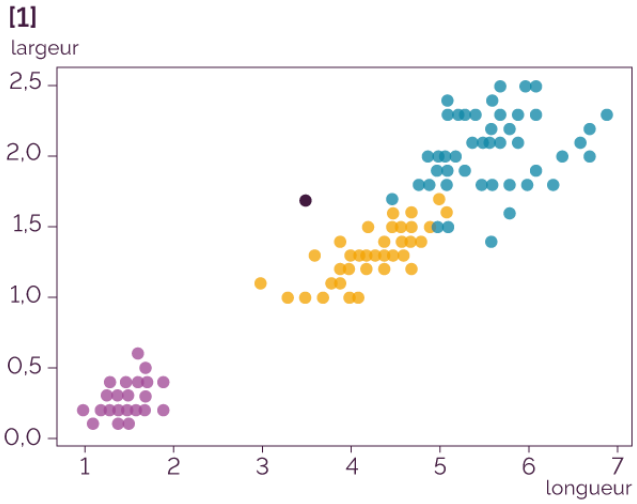
Pour utiliser l'algorithme des *k* plus proches voisins avec *k* = 5, on tape les lignes de code suivantes.

```
d=list(zip(irisData.data[:,2],irisData.data[:,3]))
model=KNeighborsClassifier(n_neighbors=5)
model.fit(d,irisData.target)
prediction=model.predict([[3.5,1.7]])
print(prediction)
```

Voici l'explication ligne par ligne.

d=list(zip(irisData.data[:,2], irisData.data[:,3]))	Extraction des données.
model=KNeighborsClassifier (n_neighbors=5)	On applique la méthode de classification knn avec un nombre de voisins égal à 5. On lui donne le nom « model ».
model.fit(d,irisData.target)	On applique cet outil au jeu de données irisData.
prediction=model.predict ([3.5,1.7]])	On demande alors la prédiction pour une mesure (3.5,1.7).
print(prediction)	On affiche ensuite cette prédiction.

À l'exécution, on obtient le graphique suivant, où le numéro de la famille apparait en haut à gauche.



L'algorithme classe ainsi la nouvelle entrée comme faisant partie de la famille 1, c'est-à-dire Versicolore (points jaunes).

COMMENT AS-TU TROUVÉ CE COURS ?

Évalue ce cours !

Note 4.6 / 5. Nombre de vote(s) : 41

Découvrez le soutien scolaire en ligne avec myMaxicours

Le service propose une plateforme de contenus interactifs, ludiques et variés pour les élèves du CP à la Terminale. Nous proposons des univers adaptés aux tranches d'âge afin de favoriser la concentration, encourager et motiver quel que soit le niveau. Nous souhaitons que chacun se sente bien pour apprendre et progresser en toute sérénité !

EXERCE TOI EN T'ABONNANT

Fiches de cours les plus recherchées

NSI

L'algorithme de recherche dichotomique dans un tableau trié

NSI

Résoudre un problème avec un algorithme glouton

Inscrivez-vous à notre newsletter !

E-mail* - Ex. contact@mymaxicours.com S'INSCRIRE

J'accepte de recevoir les actualités et des communications de la part de myMaxicours.

Votre adresse e-mail sera exclusivement utilisée pour vous envoyer notre newsletter. Vous pourrez vous désinscrire à tout moment, à travers le lien de désinscription présent dans chaque newsletter. Pour en savoir plus sur la gestion de vos données personnelles et pour exercer vos droits, vous pouvez consulter **notre charte**.

PAIEMENT SÉCURISÉ

CONTACT

01 49 08 38 00 notre équipe est joignable pour répondre à toutes vos questions du lundi au vendredi de 8 h 30 à 17 h 30.

CONTACTEZ-NOUS

7 JOURS D'ESSAI GRATUIT

Une période d'essai gratuite sur tous nos abonnements en ligne pour tester avant de vous décider !

JE TESTE MYMAXICOURS

CYCLES

Primaire | Collège | Lycée

COURS PAR CLASSE

CP | CE1 | CE2 | CM1 | CM2 | 6e | 5e | 4e | 3e | Seconde | Seconde pro | Première | Première pro | Première techno | Terminale | Terminale pro | Terminale techno

EXAMENS

Brevet | Baccalauréat | Bac pro

MATIERES

Mathématiques | Français | Technologie | Physique Chimie | SVT | Anglais | Espagnol | Allemand | Histoire | SES | Sciences | Philosophie | EMC | Histoire-Géographie géopolitique et sciences politiques | Géographie | Questionner le monde

SOUTIEN SCOLAIRE | PROFS EN LIGNE | COACHING PARENTAL ATELIERS

AIDE

Contactez-nous FAQ

A PROPOS

Qui sommes-nous ? Blog Entreprise Etablissements Scolaires Nos offres

NOUS SUIVRE

DÉCOUVREZ LES AUTRES SITES DU GROUPE

Editions Nathan | Editions Retz | Le Robert | Meet in class |
Lonely Planet | Lisez.com | Atelier des auteurs | Lizzie

Mentions légales Charte Résilier votre abonnement CGV Paramètres des cookies

Copyright © myMaxicours 2023